MOBILE APP DESIGN DOCUMENT

Study Buddy

An Android study productivity app: focus timer, task manager, progress tracker and motivation in one place.

AUTHOR
Hanan Sahin

VERSION
2.0 (updated November 2026)

ORIGIN
Mobile Application Development, Fall 2025

STATUS
Working app, in active development

PLATFORM
Android 7.0+ (API 24), Kotlin & Jetpack Compose

SOURCE
github.com/hanansahin/StudyBuddy

1 Executive Summary

StudyBuddy is an Android app that helps students build better study habits. It puts a **focus timer**, a **task manager**, a **progress tracker** and a **motivation screen** behind one bottom-navigation bar, so a student can plan, focus and reflect without switching apps.

The app began as a course project and has since grown into a working, installable app that I continue to develop. This version of the document replaces the original course submission: it describes what is actually built today, what changed from the first plan, and what comes next.

4	~750	API 24+	0
main screens plus a welcome screen	lines of Kotlin, 100% Compose UI	Android 7.0 and newer	accounts needed; data stays on device

2 Purpose and Scope

Students struggle with procrastination, disorganization and lack of focus. StudyBuddy offers a small, distraction-free toolkit that addresses all three in one place. The scope is deliberately narrow: a single-user, offline-first app (only the quote feature uses the network), with no sign-in and no ads.

3 Features and Status

Feature	What it does today	Status
Welcome screen	Gradient start screen with a “Let’s Start” button; removed from the back stack once used.	Built
Focus timer	25-minute countdown with Start / Pause and Reset.	Built
Tasks	Add, complete and delete tasks. Newest first. Saved on the device, so they survive closing the app.	Built
Progress	Animated progress ring plus total and completed task counts.	Built
Motivation	Fetches a random quote and author from the ZenQuotes web API; “New Quote” button loads another.	Built
Turkish localization	Translate the interface strings.	In progress
Room / SQLite storage	Replace the JSON file with a proper database.	In progress
Mobile security review	Documented findings and fixes for the app.	In progress
Study-time tracking	Count completed focus sessions and total study hours.	Planned
Break timer & notifications	Pomodoro breaks and alerts when a session ends.	Planned

> **What changed from the original plan.** The first document promised study-hour tracking, break sessions and notifications. Those proved larger than a first-semester build, so version 1 focused on getting the four core screens solid. They remain on the roadmap (section 14).

4 Target Audience and Personas

Primary users are high-school and college students, adult learners and anyone preparing for a certification.

User type	Goal	Feature used	Expected outcome
University student	Manage several assignments	Tasks	Checks off finished work
High-school student	Prepare for the SAT	Timer	Builds steady study intervals
Professional learner	Study after work	Progress	Sees improvement at a glance
Casual learner	Stay focused daily	Motivation	Feels encouraged to return

Sarah, 19
College freshman who struggles with procrastination and wants to stay on track.

David, 24
Studying for a professional exam while working part-time.

Leila, 17
Preparing for the SAT and looking for structure in her schedule.

5 Design Goals

- **Simple:** four screens, one tap apart, no setup or account.
- **Focused:** every screen has one job and little else on it.
- **Motivating:** visible progress and a fresh quote give a reason to come back.
- **Private:** tasks never leave the phone.

6 System Architecture

The original plan described an MVC app on SQLite. The finished app uses the modern Android pattern instead: **Compose UI → ViewModel → Repository**, with state exposed as Kotlin `StateFlow`. This keeps screens free of storage and network code.

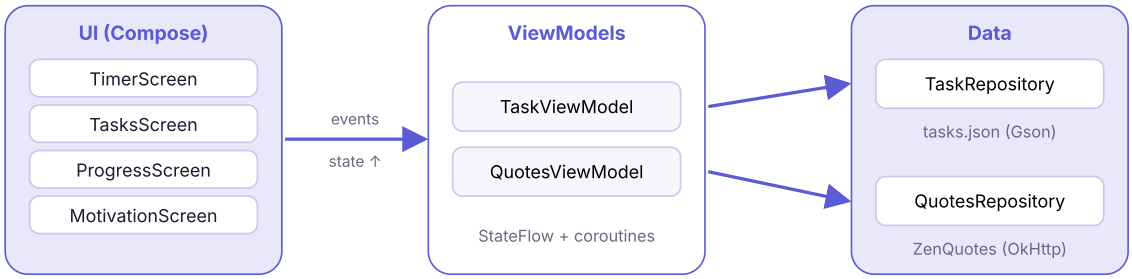


Figure 1: Layered architecture. The Timer screen keeps its own local state and does not yet use a ViewModel.

Layer	Technology
Language / UI	Kotlin, Jetpack Compose, Material 3, bundled Poppins and Inter fonts
Navigation / state	Navigation Compose (bottom bar), ViewModel, coroutines, `StateFlow`
Storage / network	Gson saving `tasks.json` in app storage; OkHttp calling ZenQuotes
Build	Gradle (Kotlin DSL), compile/target SDK 36, min SDK 24

7 Use Cases

The diagram shows what a student can do. Solid items are in the app; the dashed item is planned.

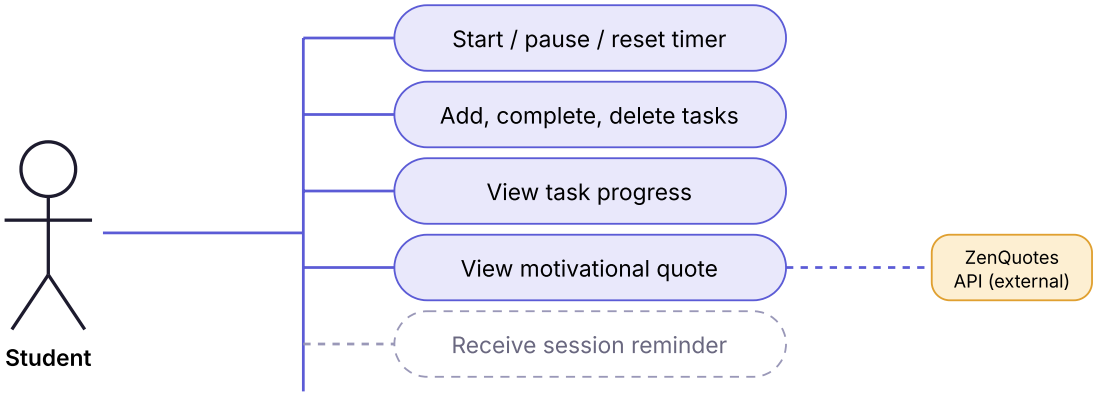


Figure 2: Use case diagram.

8 From Wireframe to Working App

The original wireframes (left) set out the four-screen layout. The finished app (below) keeps that structure. Where it differs: the Timer opens at a large countdown with Pause and Reset, Progress uses a ring instead of bars, and the Motivation screen shows a live quote with its author.

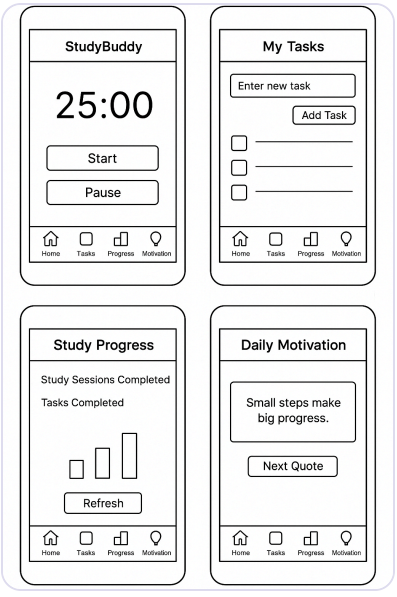


Figure 3: Original wireframes

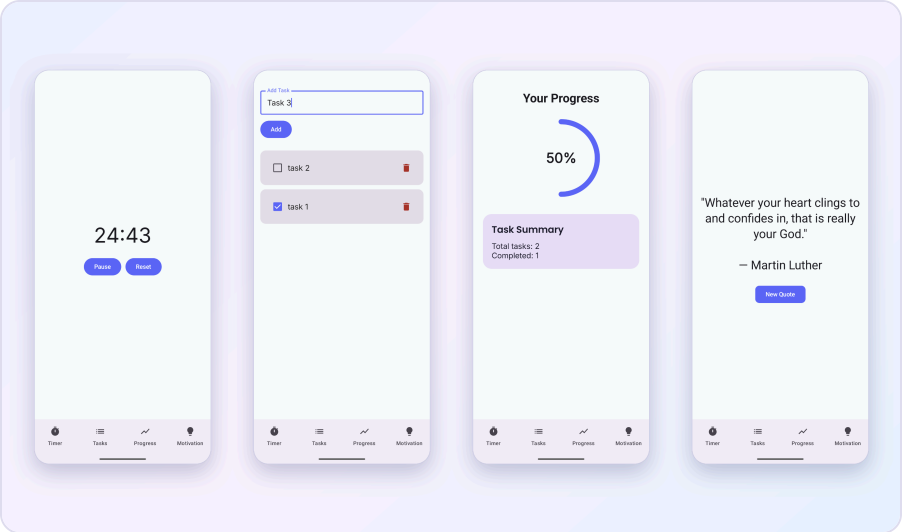
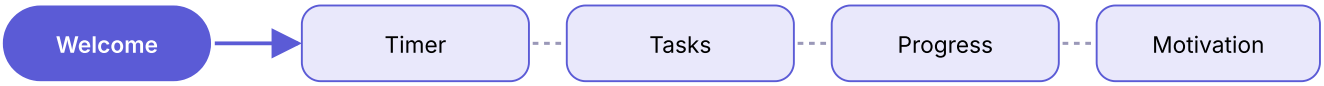


Figure 4: Final app. Timer, Tasks, Progress and Motivation.

Design details. Material 3 with an indigo accent; cards with rounded corners; red delete icon for destructive actions; progress ring animates as tasks are completed.

9 UX and Navigation Flow

The app opens on a welcome screen. Tapping **Let's Start** goes to the Timer and removes the welcome screen from the back stack, so Back exits the app rather than returning to it. From there the bottom bar switches between the four screens and restores each one's state.



Data flow example. Ticking a task in *Tasks* calls `TaskViewModel.toggleCompleted`, which updates the saved file and reloads the list. *Progress* observes the same ViewModel, so its ring updates automatically.

10 Non-Functional Requirements

Area	Target	Where it stands
Performance	Timer accurate to about ±1 second per session	Implemented with a 1 s coroutine tick; not yet measured
Usability	Simple navigation, readable type, few distractions	Met: four screens, Poppins / Inter fonts bundled
Reliability	Tasks survive closing the app	Met for tasks. Timer state resets when you leave the screen.
Compatibility	Originally Android 9+	Now Android 7.0+ (API 24)
Maintainability	Organized in modules	Packages by layer: data, models, nav, ui, viewmodels
Privacy	No account, no tracking	Met. Only network call is the quote request.

11 Development Timeline

The original plan was a seven-week course schedule. Here is how the work actually unfolded.

- **Sep 2025** · Proposal, design document, personas and wireframes
- **Fall 2025** · Compose UI, bottom navigation, timer, task list with on-device saving, progress ring
- **Fall 2025** · Motivation screen with live quotes from ZenQuotes; welcome screen
- **Now** · Turkish localization, Room/SQLite migration and a mobile security review
- **Next** · Study-time tracking, break timer and notifications

12 Competitive Analysis

	Forest	Focus To-Do	StudyBuddy
Focus timer	Yes, gamified	Yes	Yes
Task list	No	Yes	Yes
Motivation content	Virtual trees	No	Live quotes
Account needed	Optional	Optional	No
Turkish interface	Not a focus		In progress

StudyBuddy does not try to out-feature these apps. Its angle is a small, calm, private toolkit that combines timing, tasks and encouragement, with Turkish-speaking students as an under-served audience. *(Competitor features are summarized from general knowledge of the apps and should be re-checked before publishing.)*

13 Challenges, Limitations and Lessons

Known limitations

- The timer resets if you switch screens, and it does not run in the background or notify you when it ends.
- The timer is fixed at 25 minutes; there is no break mode.
- Progress counts tasks only; it does not yet track study sessions or hours.
- Tasks are stored in a JSON file that is rewritten on every change, which does not scale and is being replaced by Room.
- If the quote request fails, the error text is shown in place of a quote. A friendlier fallback is needed.
- The bottom bar does not yet highlight the current screen.

Lessons learned

- Start with a thin working slice of every feature, then deepen it.
- Separating UI, ViewModel and repository made it easy to swap storage plans mid-project.
- Scoping honestly matters: cutting notifications from version 1 kept the core app reliable.

> **Security review: early observations.** Items to examine as part of the planned review: automatic backup is enabled (`allowBackup`) so the task file may be included in device backups; release builds are not yet minified or obfuscated; and tasks are stored unencrypted. The only network traffic is an HTTPS request to ZenQuotes.

14 Roadmap

Now
- Turkish localization
- Room / SQLite storage
- Mobile security review

Next
- Pomodoro breaks
- Timer that survives navigation
- Session notifications

Later
- Study-hours analytics
- Custom themes and sounds
- Optional cloud sync (Firebase)

15 Developer Notes

Building StudyBuddy taught me how a real Android app fits together: Compose state, navigation, ViewModels, coroutines, local persistence and a network API. It also taught me to keep documentation honest. Writing down what is unfinished is what turned the roadmap into my next set of tasks.

The planned security review is where this project meets my cybersecurity interests: I intend to test my own app, record the findings and publish the fixes.

16 Conclusion

StudyBuddy started as a course assignment and is now a working app with a clear architecture, a documented roadmap and an honest list of limitations. It shows the full cycle from user research and wireframes to a tested build, and it keeps improving.

17 Credits and Links

Source code	github.com/hanansahin/StudyBuddy
Quotes	ZenQuotes API (zenquotes.io)
Author	Hanan Sahin · Computer Science, University of Maryland Global Campus
Licence	See LICENSE in the repository